

Dimensions.c: parsed argv to get the modifier chain and one image path. The modifier chain rejects any unknown flags or mutually exclusive flags like -w/-h. I verified that the file exists and it is a .tga file. I pushed all of the output formatting to formatDimensions so that the CLI stays simple. I load the image via the library using formatDimensions(img, verbose, mode). formatDimensions is a helper from operations.c. I then print the string returned from formatDimensions, and free memory. I formatted this similarly to how operations.c is set up, where the program does the parsing and validation and the helper library does the logic.

Split.c: I again validated that the file exists, and is .tga. If the input is RGBA32, I iterated each row(unsigned int*), and for each pixel(unsigned char*), I copied the requested channel byte into a gray_raster image using extract_gray. If the input is gray_raster, it is just cloned 3 identical times and just considered different channels.

compare.c : i used compareImagesExact(a,b) (a and b are const RasterImage*) from the operations header to check width, height, type, bytesPerPixel, and bytesPerRow, then memcmp the full raster size, and return 1 if they are equal, else return 0. The design choice to rely on memcmp (instead of a per-pixel loop) is both simpler and faster once layouts match.

Matchsearch.c: i checked if the file was a .tga file. It launches one child process per file using fork() followed by execl() or the compare executable(w/ argument of the input image and the candidate file). The parent tracks pairs with the contents {pid, candidateName} in an array named Job. After launching children, it collects results with waitpid, checks WIFEXITED, and receives WEXITSTATUS==1 which is a match. It then pushes the match name into a StrVec vector. It then writes the report to the output file with the format that the handout specifies.

[build.sh](#): i added lines to compile dimensions.c, split.c, compare.c, matchSearch.c with the same format as what was already in the script, which was saving the paths as a variable and then using that variable to compile the path to the executable for each program.

[script03v1.sh](#): calls the build script, checks if the output folder exists, if not creates a new one, and if it does exist it deletes all files and subfolders in it. I used the find function to find all files with the .tga extension. I used find because it recursively searches the folder and returns all cases of .tga.

[script03v2.sh](#): this script is actually the same as v1 because the find function is recursive.

Assumptions and limitations.

It was unclear whether the separated color channel outputs from compare.c were supposed to be grayscale or tinted. Through my own research I realized that color channels are usually grayscale when separated so that was what I implemented.

I also had to assume where certain files go because the wording in the handout was not always consistent, for example in one part the handout refers to a 'Programs' folder which we do not have, so i had to assume that 'Applications' was the folder it was referring to.